PRACTICAL 8

Aim

To implement stock market analytics using Window-Based Analysis in Apache Spark with PySpark.

The practical focuses on loading real historical stock market data obtained from Yahoo Finance into a Spark DataFrame, examining its schema and records, grouping stock data by stock symbol, calculating average, maximum and minimum stock prices, applying Spark Window functions, calculating running averages and moving averages, determining daily stock returns and volatility, comparing stock performance, generating a summary report, and visualizing stock price trends.

Problem Definition

A financial services company processes large volumes of historical stock market data collected from multiple stock exchanges. Investors and analysts need to understand stock price movements, compare different stocks, identify trends, and analyze their performance over time.

As the volume of financial data increases, traditional data processing methods may become inefficient for performing repeated calculations and time-based analysis. Apache Spark provides distributed DataFrame operations and Window functions that can efficiently perform analytical calculations over large datasets.

The task is to load real historical stock market data from Yahoo Finance for multiple stocks into a Spark DataFrame and perform window-based stock market analysis.

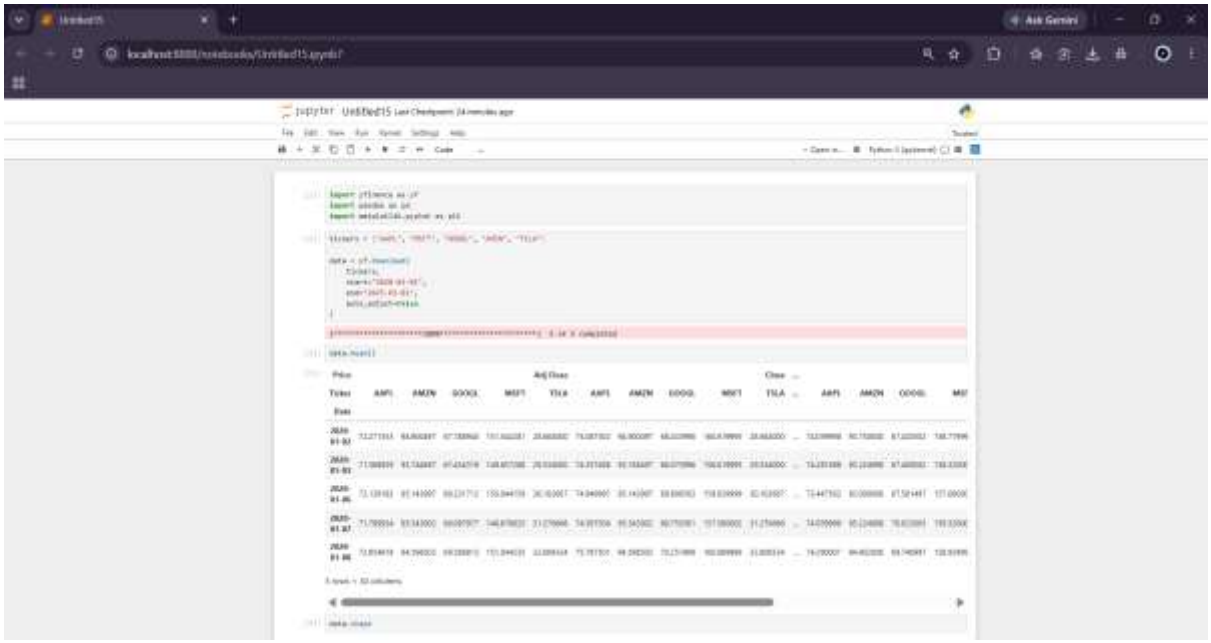
The practical includes creating a SparkSession, loading and displaying the stock dataset, examining the schema, grouping records based on stock symbols, calculating average, maximum and minimum stock prices, creating Window specifications, calculating running averages and 3-day moving averages, calculating daily stock returns using lag(), measuring stock volatility, comparing total stock returns, generating a stock performance summary report, and visualizing stock price trends.

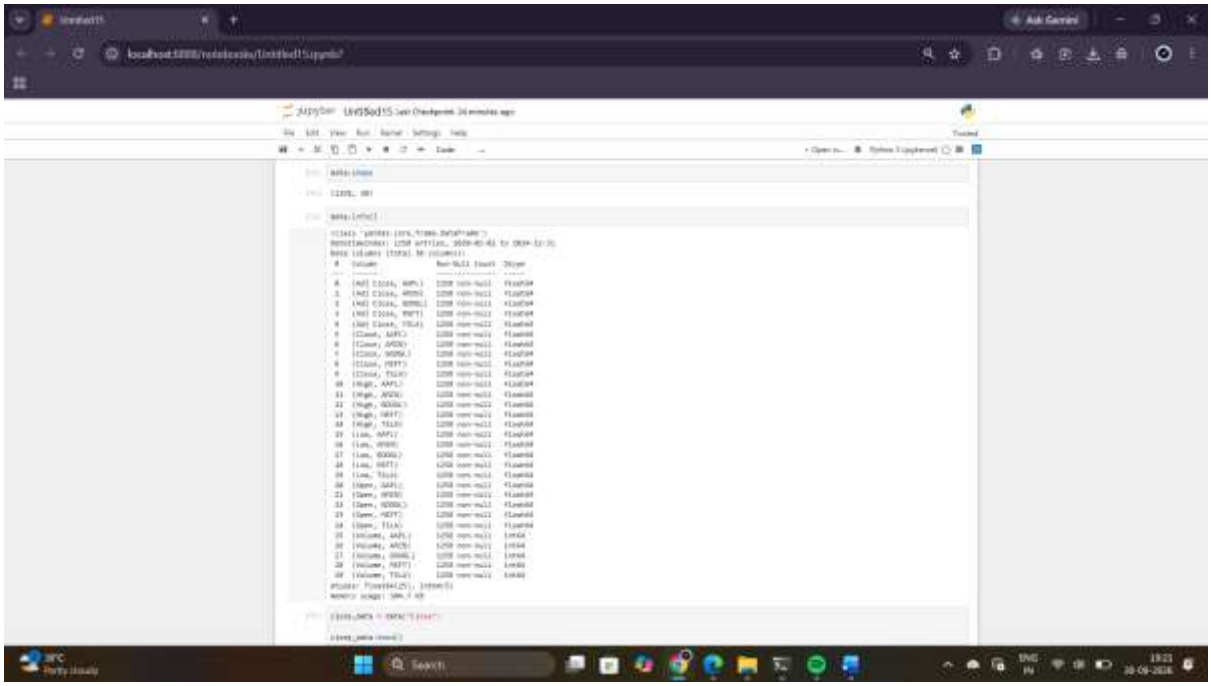
Objectives

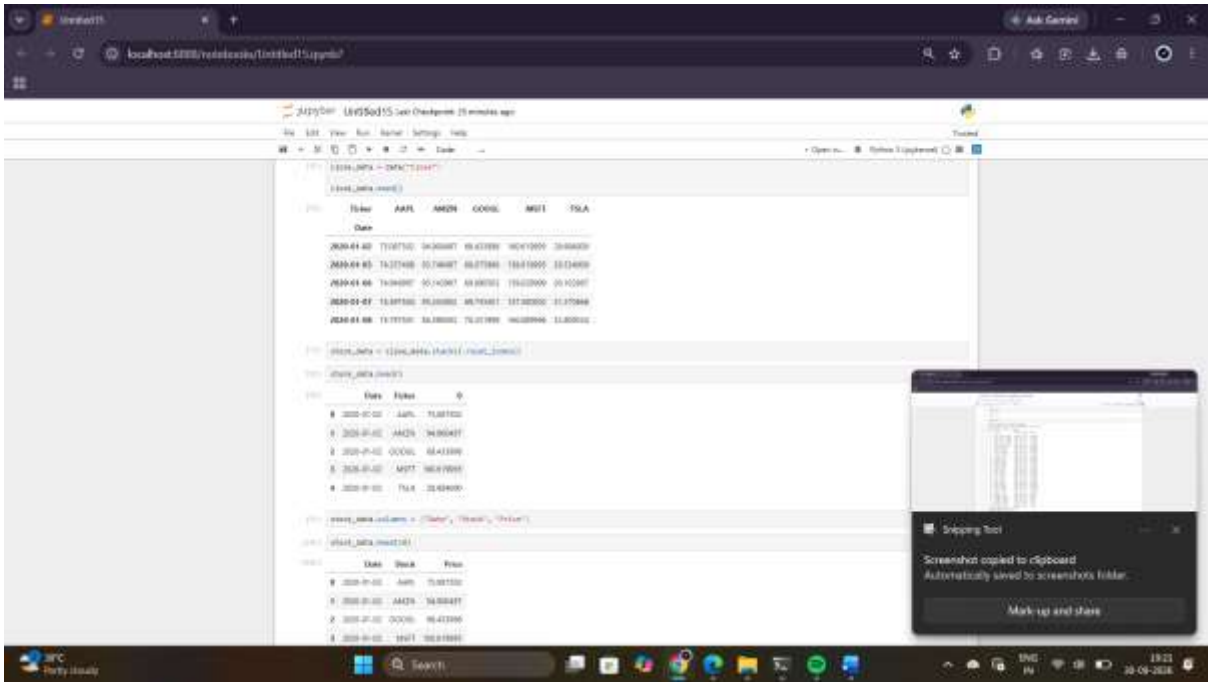
- Understand stock market analytics using Apache Spark.
- Create and configure a SparkSession.
- Obtain real historical stock market data using Yahoo Finance.
- Load multiple stock records into a Spark DataFrame.
- Display and examine the stock dataset.
- Display the schema of the DataFrame.
- Group stock records based on stock symbol.

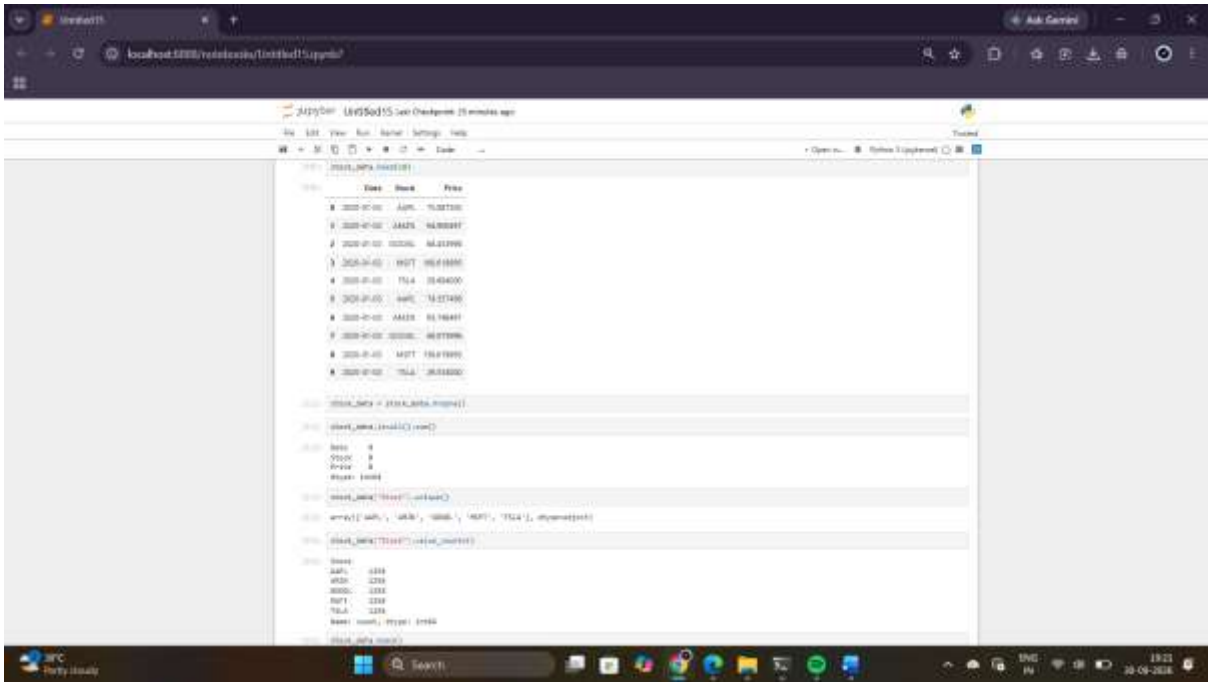
- Calculate average stock price.
- Calculate maximum stock price.
- Calculate minimum stock price.
- Create Window specifications in Spark.
- Calculate running average stock prices.
- Calculate 3-day moving averages.
- Use lag() to obtain previous stock prices.
- Calculate daily stock returns.
- Calculate stock volatility.
- Calculate total stock returns.
- Compare the performance of multiple stocks.
- Generate a stock market summary report.
- Visualize historical stock price trends.
- Understand the importance of Window functions in time-series analysis.
- Analyze stock market trends using Spark.

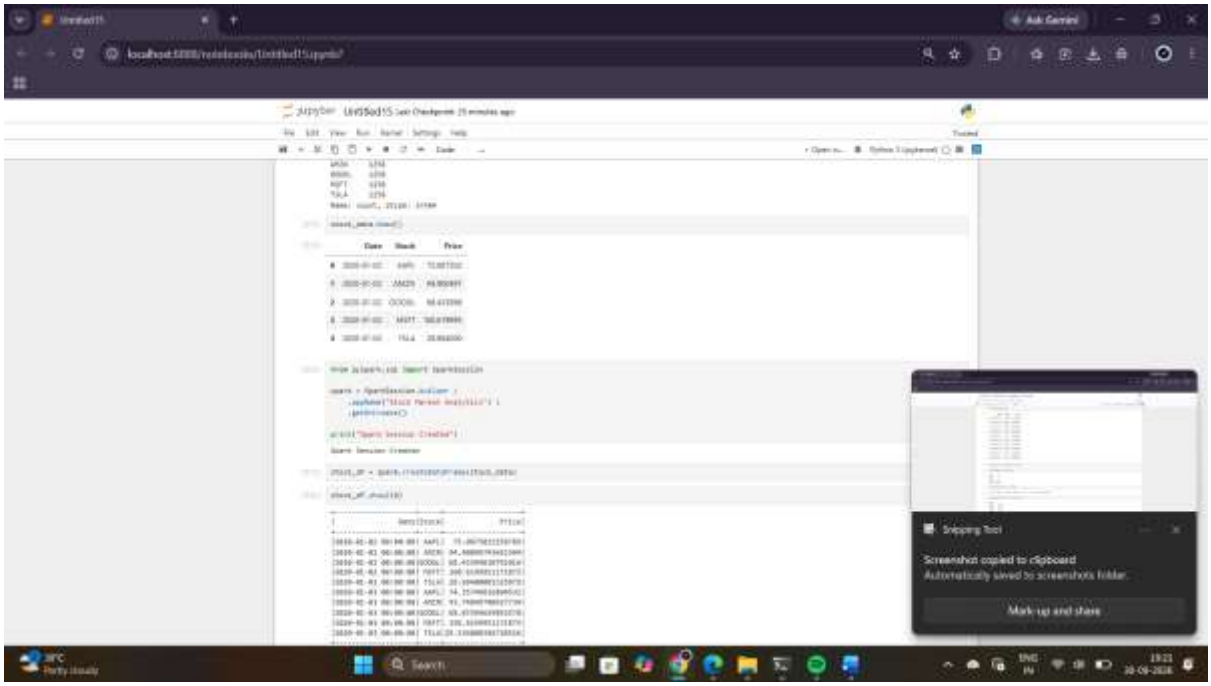
Output

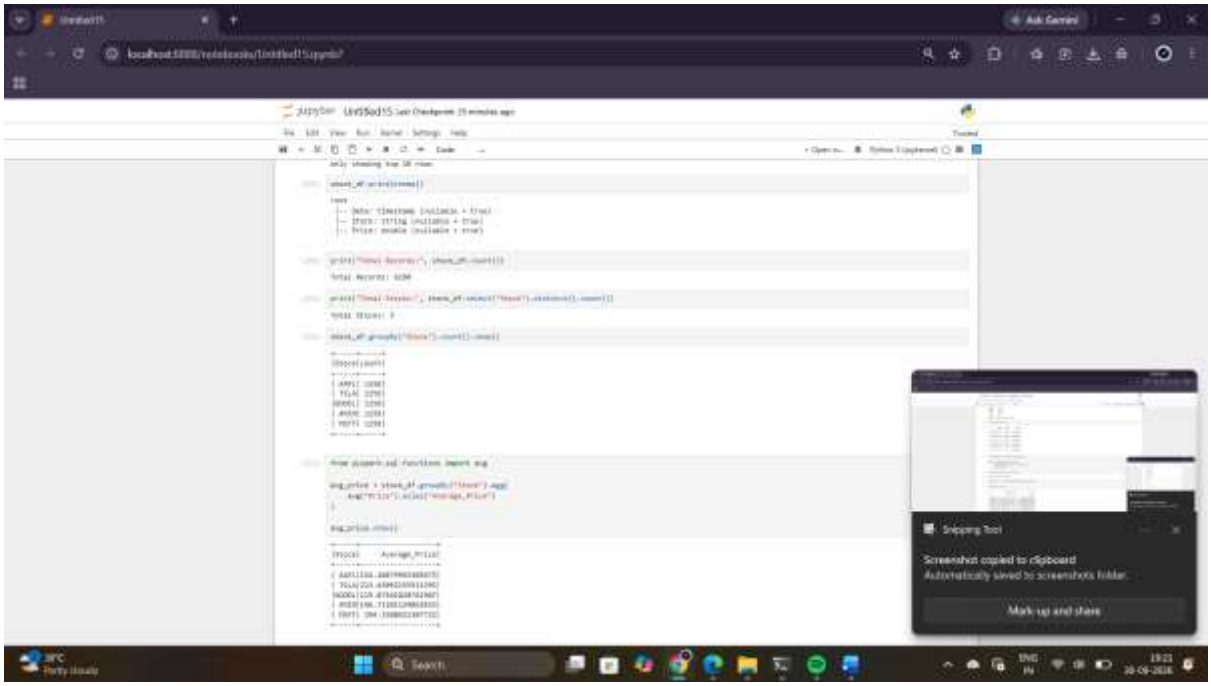


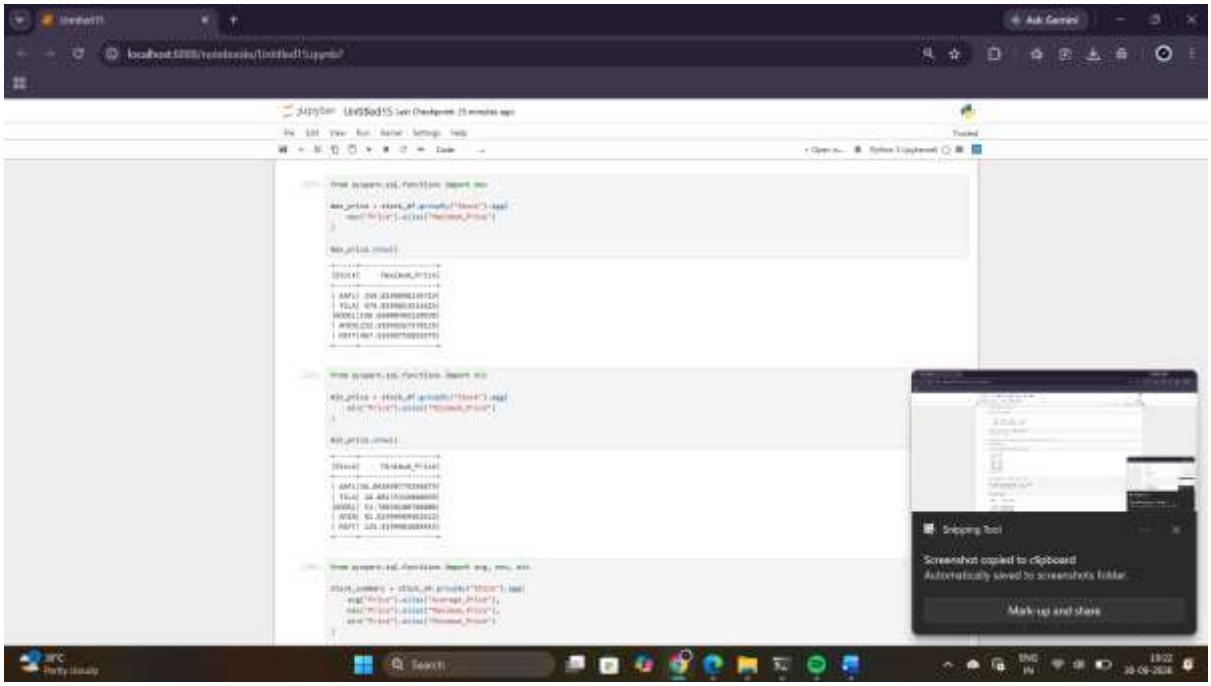


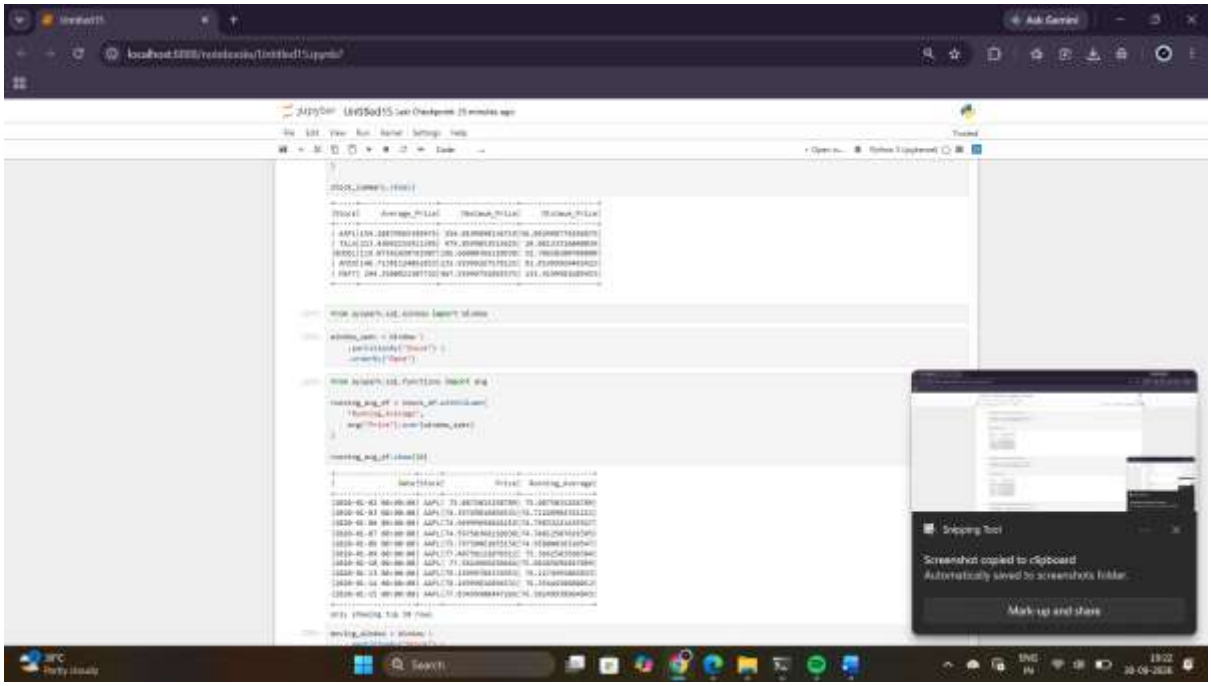


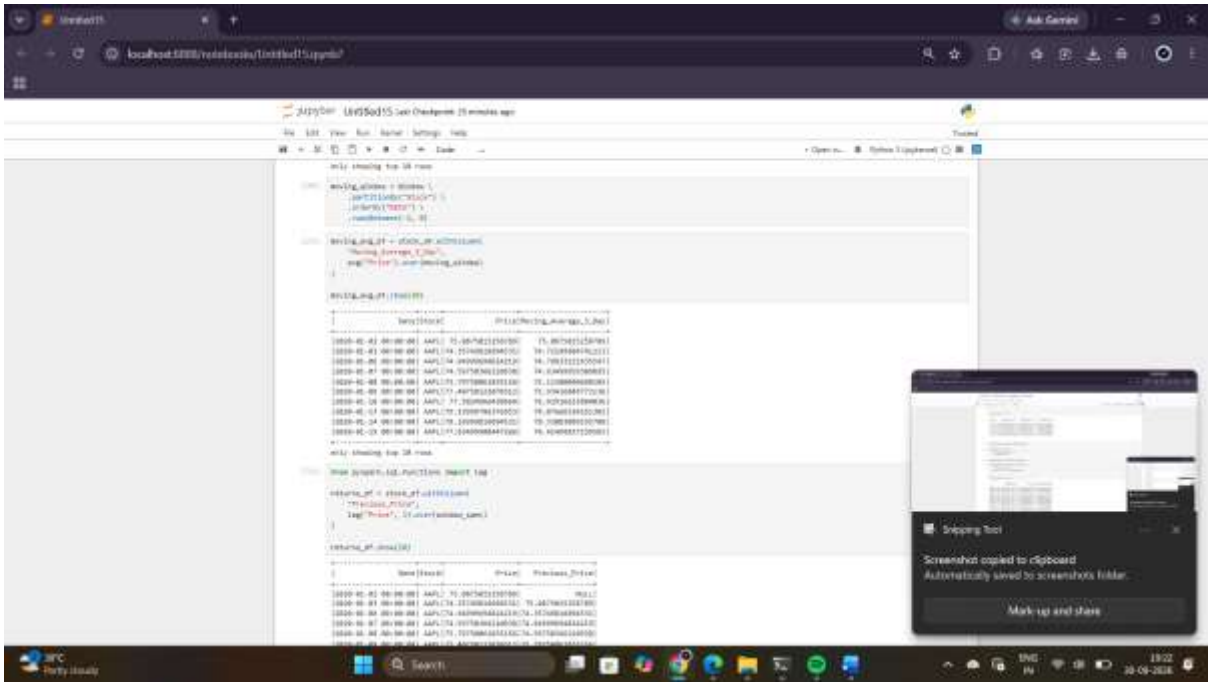


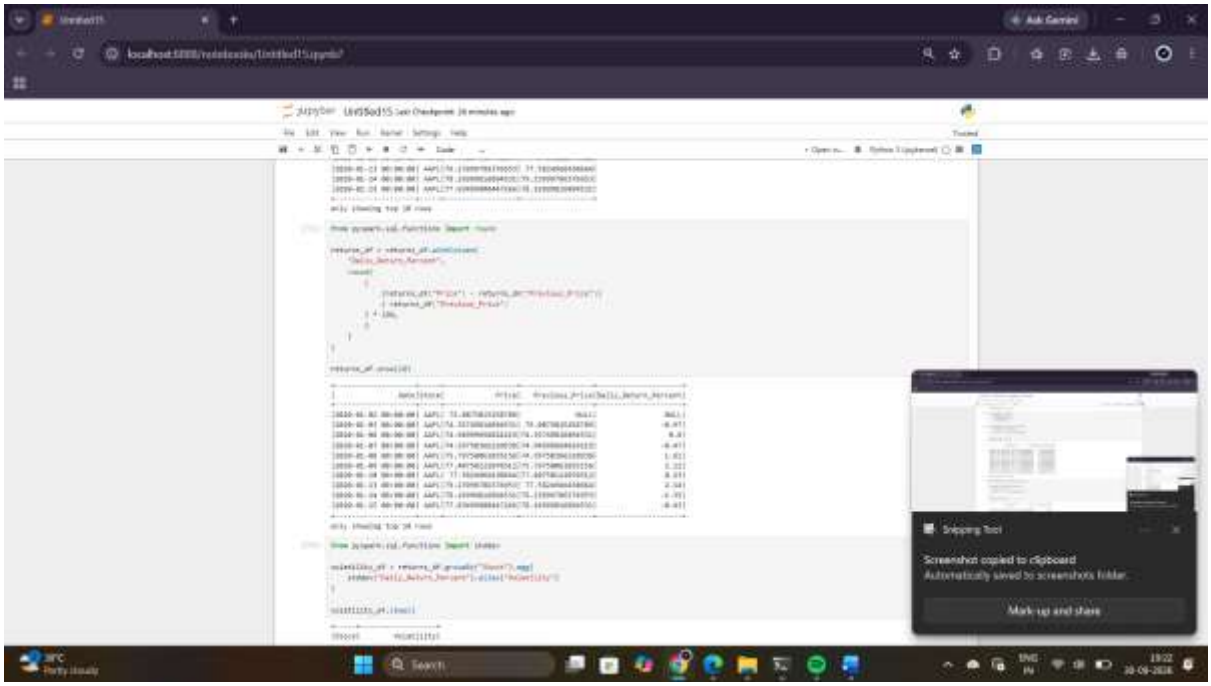


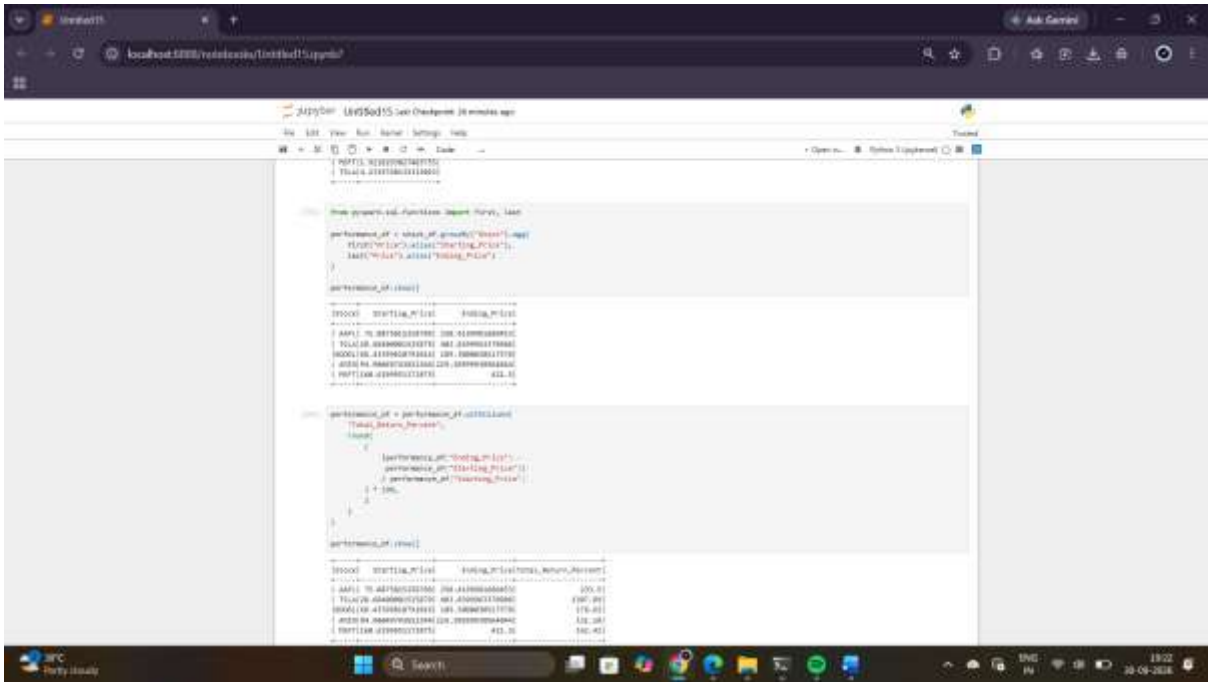

Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Mark up and share

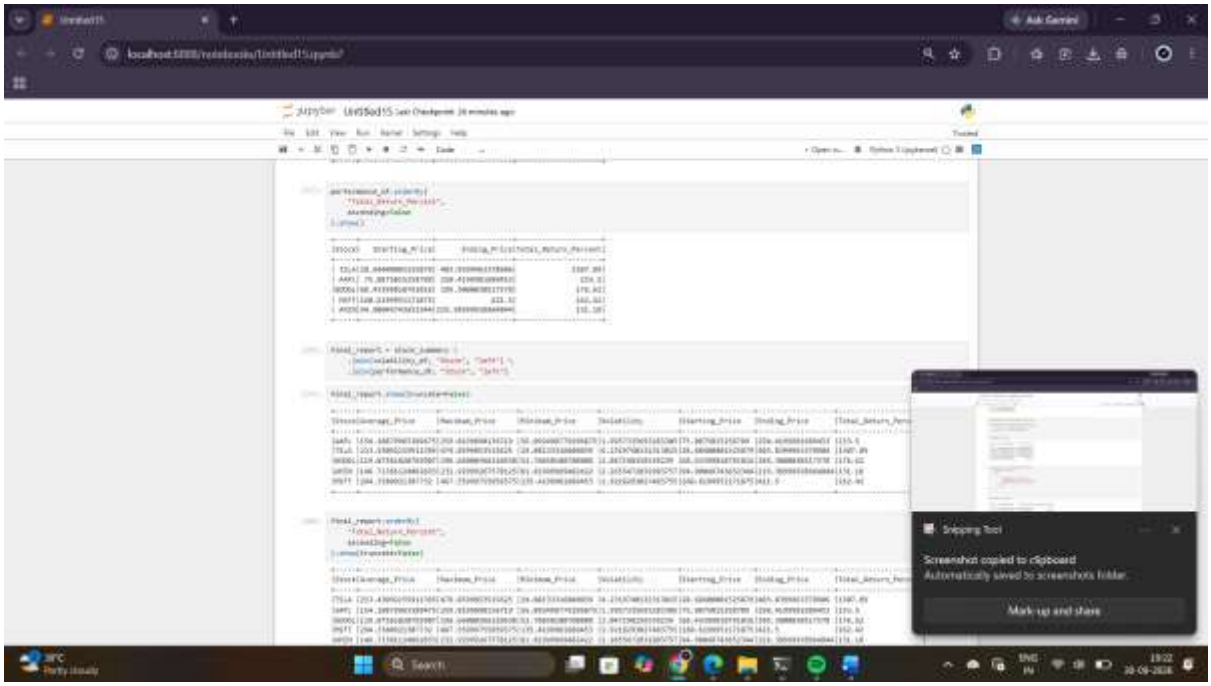

Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Mark up and share

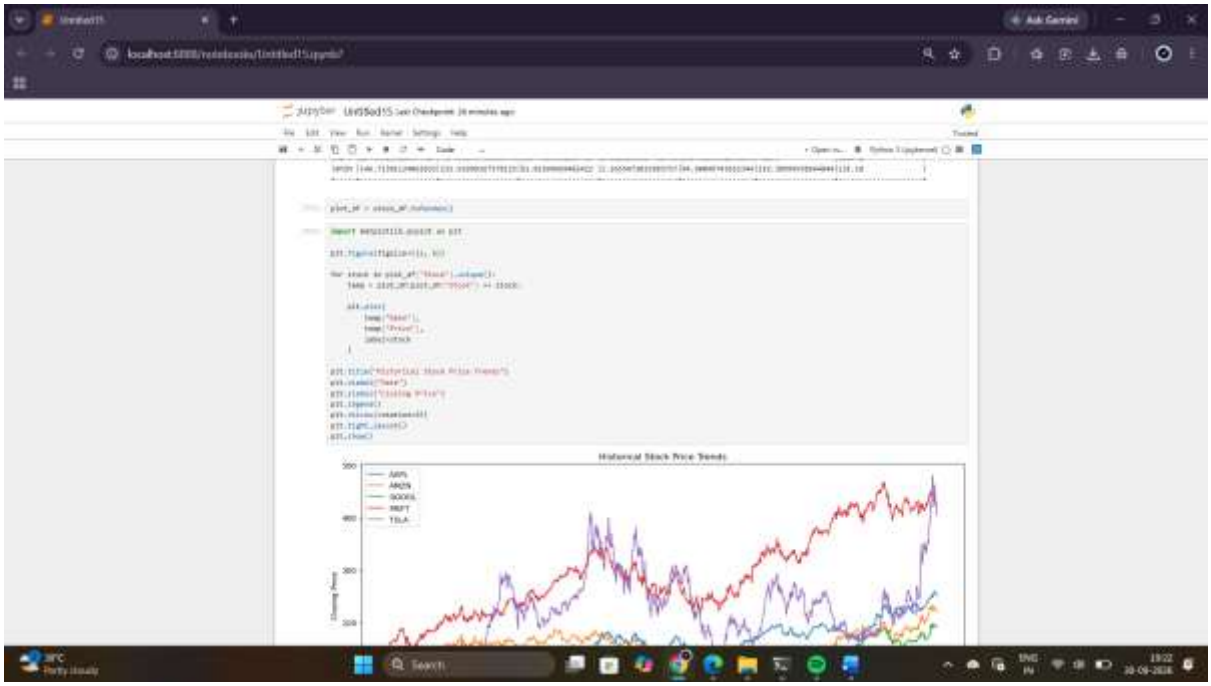


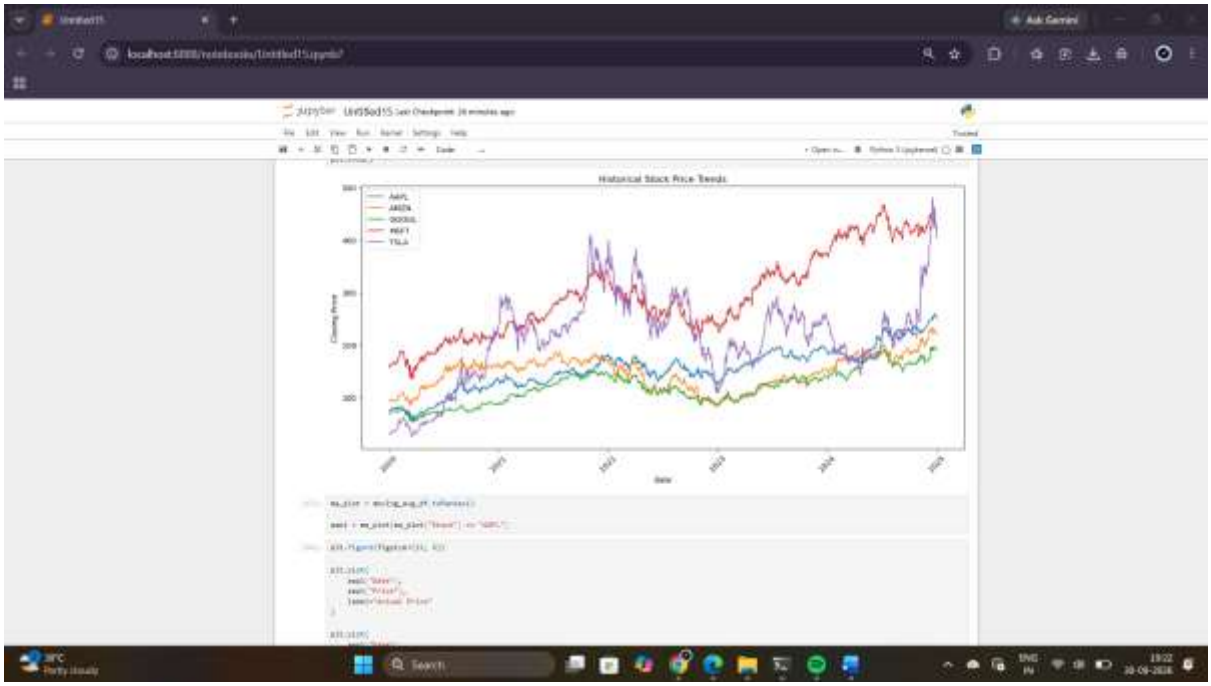


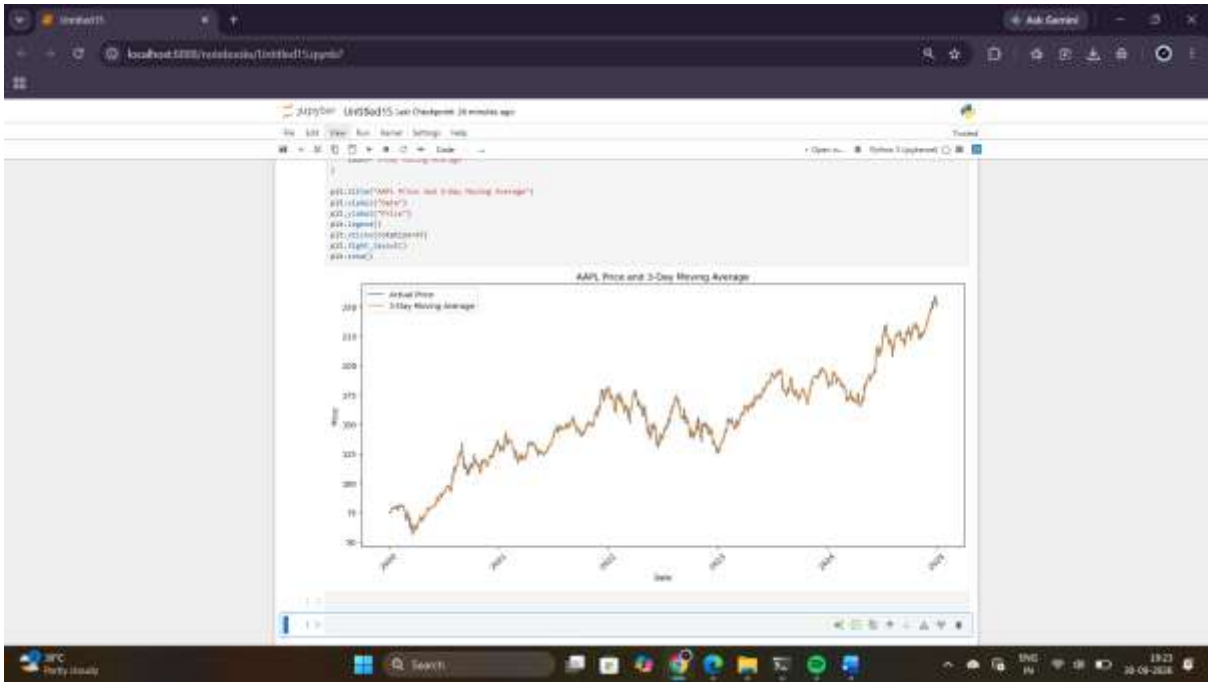

Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Mark-up and share




Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Mark-up and share


Historical Stock Price Trends
AAPL
AMZN
GOOGL
MSFT
TSLA





Key Questions / Analysis

Q1. Why is Apache Spark suitable for financial analytics?

Apache Spark is suitable for financial analytics because it can process large volumes of financial data efficiently using distributed processing.

Spark provides DataFrames, aggregation functions and Window functions that are useful for analyzing historical stock prices. It can perform calculations such as average price, maximum price, minimum price, moving averages, daily returns and volatility.

Spark can also process data in parallel, making it suitable for large-scale financial datasets.

Q2. What is the purpose of grouping stock data by stock symbol?

Grouping stock data by stock symbol separates the records of each individual stock.

For example, stocks such as AAPL, MSFT, GOOGL, AMZN and TSLA can be analyzed separately.

Grouping allows us to calculate statistics such as:

1. Average stock price
2. Maximum stock price
3. Minimum stock price
4. Stock volatility
5. Total return

In PySpark, grouping can be performed using:

stock_df.groupBy("Stock").count().show()

Q3. How does average stock price help investors?

Average stock price provides an overall measure of the typical price of a stock during a selected period.

It helps in understanding the general price level of the stock and allows investors to compare the historical price levels of different stocks.

However, average price alone does not indicate whether a stock is a good investment because other factors such as returns, volatility, company performance and market conditions also need to be considered.

Q4. Why are maximum and minimum prices important?

Maximum and minimum prices show the highest and lowest prices recorded by a stock during a particular period.

They help in understanding:

1. The historical price range.
2. Price fluctuations.
3. High and low price levels.

4. Changes in stock prices over time.
5. The overall movement of the stock.

These values can also be used along with other metrics to analyze historical stock trends.

Q5. How can Spark support real-time stock market monitoring?

Apache Spark can support stock market monitoring by processing continuously generated market data and performing analytical operations on incoming records.

Spark can be used to:

1. Process large volumes of incoming stock data.
2. Calculate price changes and returns.
3. Track moving averages.
4. Detect unusual price movements.
5. Calculate volatility.
6. Compare stock performance.
7. Generate analytical results for monitoring systems.

For real-time applications, Spark's Structured Streaming can process continuously arriving data, allowing analytical calculations to be performed as new market records are received.

Supplementary Problems

1. Calculate daily stock returns.
2. Identify top-performing stock over a period.
3. Compute moving average for stock prices.
4. Compare stock volatility across companies.
5. Visualize stock price trends using charts.

Conclusion

The Stock Market Analytics Using Window-Based Analysis practical was successfully completed using PySpark and real historical stock market data obtained from Yahoo Finance.

The stock data for multiple companies was loaded into a Spark DataFrame and its records and schema were examined. The data was grouped according to stock symbols, and average, maximum and minimum stock prices were calculated.

Spark Window functions were then used for time-series analysis. Running averages and 3-day moving averages were calculated to analyze stock price trends. The lag() function was used to obtain previous stock prices, which enabled the calculation of daily stock returns.

Stock volatility was calculated using the standard deviation of daily returns, and starting price, ending price and total return percentage were used to compare the historical performance of different stocks. A final summary report was generated containing the major stock performance metrics.

The practical demonstrated how Spark DataFrames, aggregation functions, Window functions, lag(), moving averages and distributed processing can be used for analyzing large-scale financial datasets and identifying historical stock market trends.